

Apuntes 5

Indexing

Sin índices, se hace un **scan** que es buscar datos, lo que implica recorrer todos los elementos $O(N)$, lo que es lento en grandes volúmenes

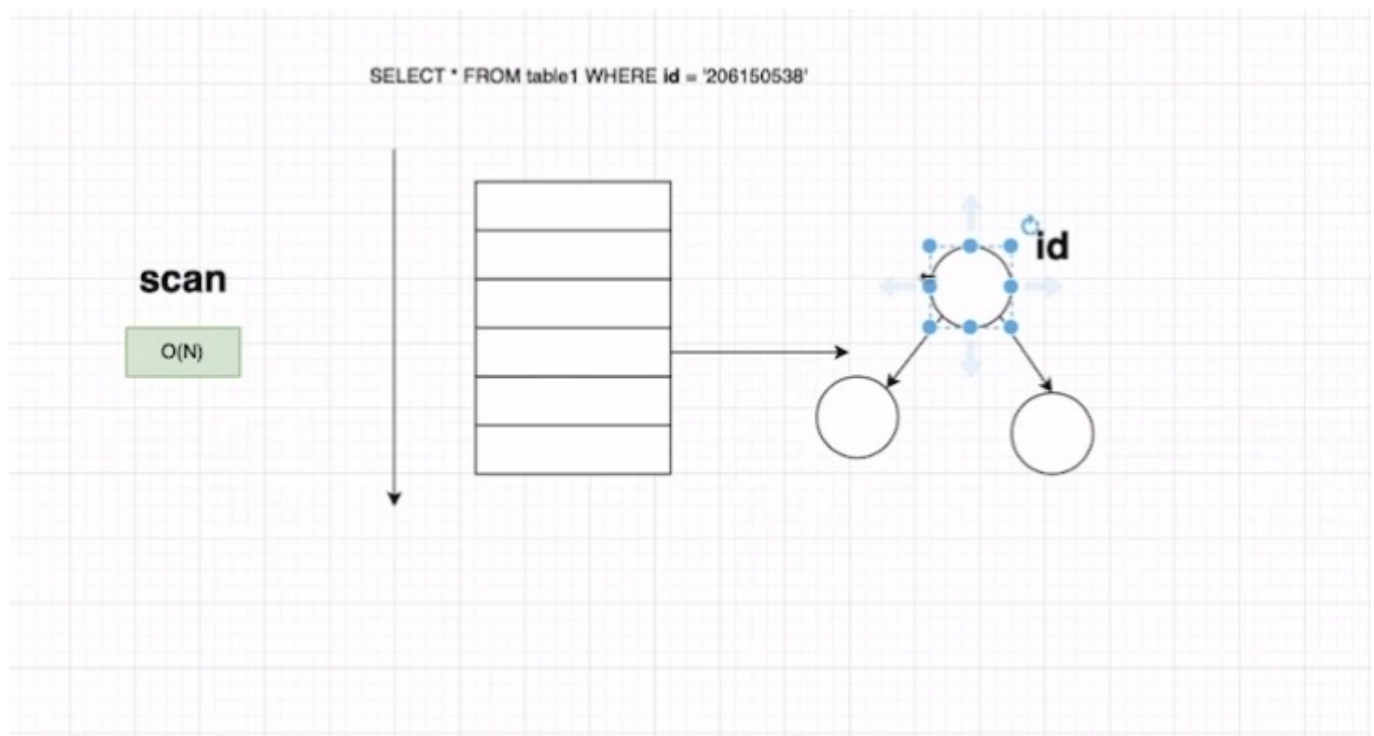
Los índices son estructuras de datos que optimizan:

- Búsqueda
- Inserción
- Borrado

Al insertar, el motor no hace un simple append: recorre la estructura para ubicar el lugar correcto

En sistemas con muchas escrituras, conviene desactivar temporalmente la indexación para mejorar el rendimiento

Luego se reactiva el índice y se reorganiza la estructura para mantener eficiencia



Clustered Index

Es una estructura de datos física que se crea dentro del archivo de datos de la base

Solo puede existir un clustered index por tabla, porque define cómo se almacenan los datos físicamente

Elegir las columnas para el clustered index requiere análisis del workload: se seleccionan las columnas más usadas en consultas SELECT

Su objetivo es **optimizar** el acceso a datos que se consultan frecuentement

Non Clustered Index

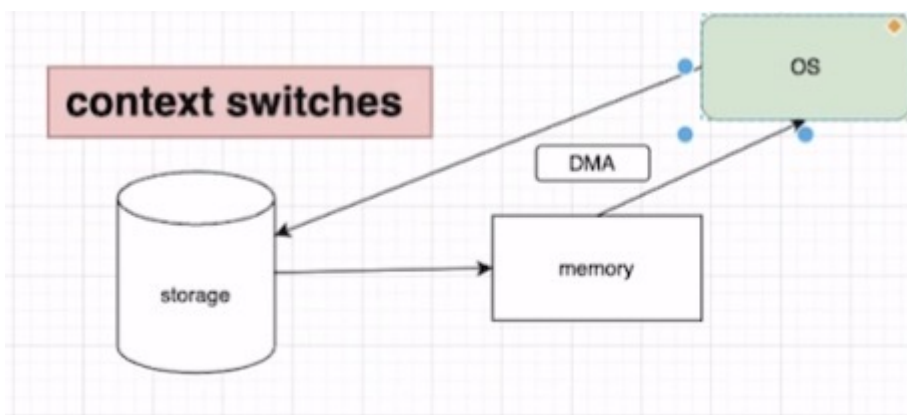
Es una estructura de datos separada del archivo principal de datos

A diferencia del clustered index, no define el orden físico de los datos en disco

Puede haber varios non-clustered indexes por tabla

Contiene una copia ordenada de las columnas indexadas y referencias (punteros) a las filas reales en el archivo de datos

- Se usa para acelerar búsquedas en columnas que no forman parte del clustered index
- Ideal para consultas frecuentes sobre columnas específicas, especialmente cuando se combinan con filtros (WHERE) o agrupamientos (GROUP BY)



Hash Index

Un Hash Index usa una función hash para convertir una clave en un identificador de bucket

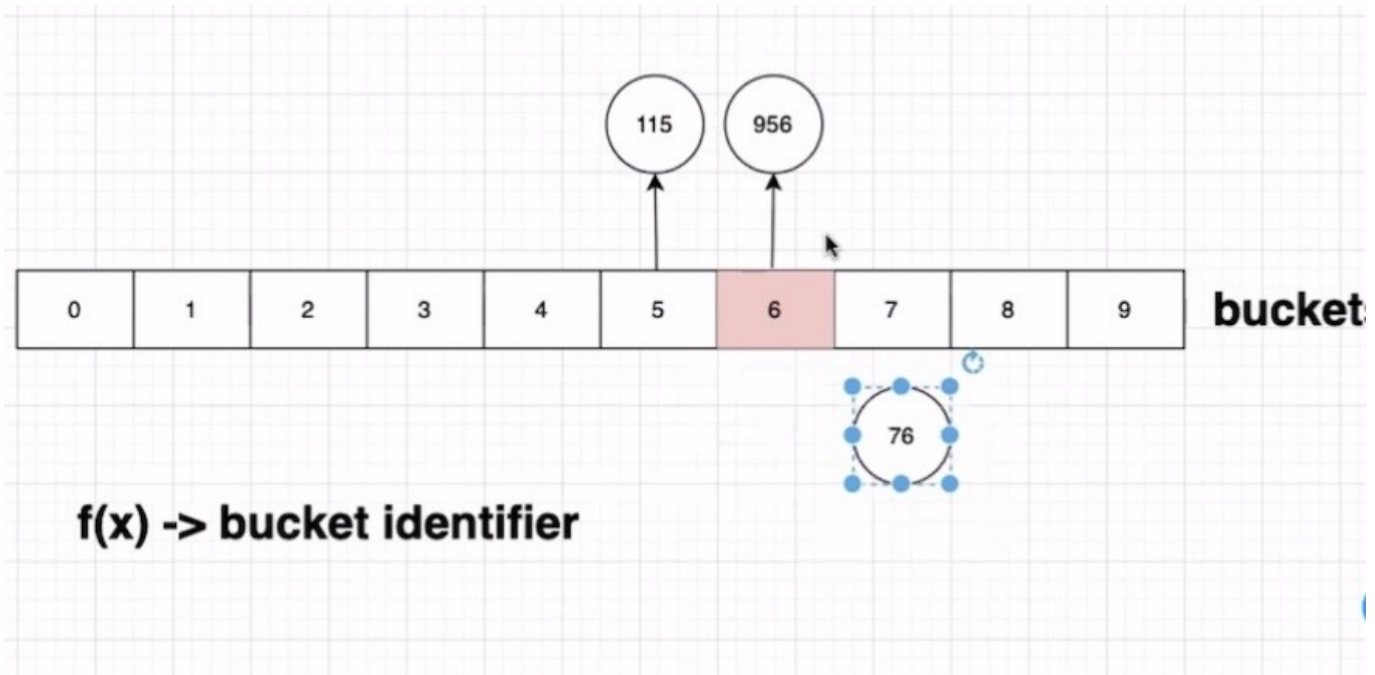
Los buckets son contenedores numerados donde se almacenan los datos indexados

Si varios valores caen en el mismo bucket se le conoce como **colisión**, se aplican estrategias como:

- Buscar buckets cercanos disponibles
- Usar estructuras internas (listas, árboles, índices multibinarios)

Este tipo de índice es muy eficiente para búsquedas exactas, pero no sirve para rangos ni ordenamientos

El diseño debe considerar la distribución de los datos para minimizar colisiones y mantener el rendimiento



B-Tree, B+Tree e índices con INCLUDE

Las estructuras B-Tree y B+Tree permiten búsquedas eficientes al reducir la cantidad de operaciones necesarias

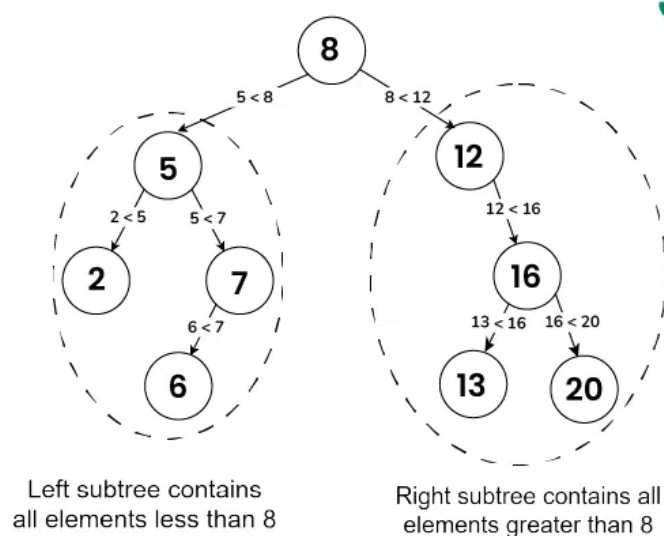
En ciertos casos, pueden ofrecer tiempos de búsqueda cercanos a **O(1)** si están bien balanceadas

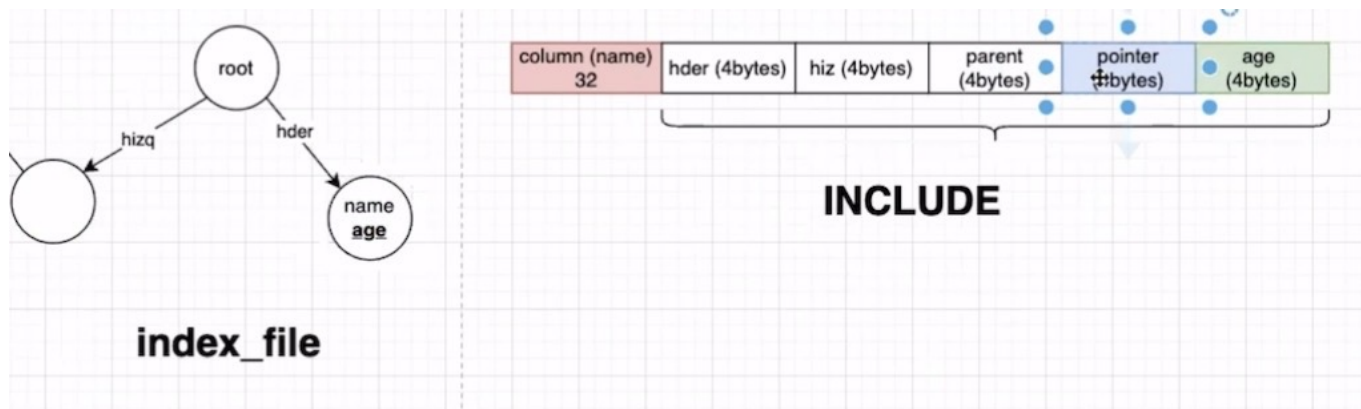
La cláusula **INCLUDE** permite agregar columnas adicionales al índice que no forman parte de la clave principal

Esto mejora el rendimiento porque el dato está listo para usarse sin acceder al registro completo

Sin embargo, usar INCLUDE aumenta el consumo de memoria, CPU y almacenamiento, por lo que debe aplicarse con criterio

Binary Search Tree





Expression Index

Un Expression Index guarda el resultado de aplicar una función o expresión sobre una columna (por ejemplo, `lower(name)`)

Esto permite que las consultas que usan esa expresión (como `WHERE lower(name) = 'nero'`) sean mucho más rápidas, ya que:

Evitan calcular la expresión en cada fila durante la búsqueda.

Es útil cuando una expresión se repite frecuentemente en consultas

Mejora el rendimiento, pero aumenta el uso de almacenamiento porque guarda resultados adicionales

